

DFF File Format

Dolphin FIFO log, version 6

Reference implementation: [crates/dff/src/lib.rs](https://crates.io/crates/dff)

1 Overview

A `.dff` file is a recording of the GameCube/Wii GPU command stream. It starts with a snapshot of the GPU state at the moment recording began (BP, CP and XF registers plus TMEM), followed by the raw FIFO bytes of each frame and the guest RAM the FIFO reads. That is enough to replay the frames without running any CPU emulation.

All integers are little-endian. The structs are packed, so the `u64` offsets sit at 4-byte boundaries instead of natural alignment. Offsets are absolute file offsets. Always take section positions from the header, not from the layout below.

Section	Contents
Header (128 B)	magic, version, section offsets, flags, game ID
Frame table	<code>frameCount</code> × 64 B frame descriptors
<code>bpMem</code>	256 × <code>u32</code> , the BP raster-state registers (TEV, PE, texture units)
<code>cpMem</code>	256 × <code>u32</code> , the CP (command processor) registers
<code>xfMem</code>	4096 × <code>u32</code> , XF (transform unit) memory
<code>xfRegs</code>	88 × <code>u32</code> , the XF registers (xfmem 0x1000 to 0x1057)
<code>texMem</code>	1 MiB TMEM snapshot
Frame payloads	per frame: FIFO bytes, memory-update table, update payloads

Table 1: File layout as emitted by the gecko writer. Readers must follow header offsets.

2 Header

128 bytes at offset 0.

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+10	+11
0x00	fileId				fileVersion				minLoaderVersion			
0x0C	bpMemOffset								bpMemSize			
0x18	cpMemOffset								cpMemSize			
0x24	xfMemOffset								xfMemSize			
0x30	xfRegsOffset								xfRegsSize			
0x3C	frameListOffset								frameCount			
0x48	flags				texMemOffset							
0x54	texMemSize				mem1Size				mem2Size			
0x60	gameId								padding			
0x6C	...											
0x78	...											

Offset	Size	Field	Description
0x00	4	fileId	Magic 0x0D01F1F0
0x04	4	fileVersion	Currently 6
0x08	4	minLoaderVersion	Reject the file if this exceeds the loader's supported version
0x0C	8	bpMemOffset	Absolute offset of BP register block

Offset	Size	Field	Description
0x14	4	bpMemSize	Element count in u32 units (256)
0x18	8	cpMemOffset	CP register block
0x20	4	cpMemSize	256
0x24	8	xfMemOffset	XF memory block
0x2C	4	xfMemSize	4096
0x30	8	xfRegsOffset	XF register block
0x38	4	xfRegsSize	88
0x3C	8	frameListOffset	Frame descriptor table
0x44	4	frameCount	Number of frame descriptors
0x48	4	flags	Bit 0: IS_WII
0x4C	8	texMemOffset	TMEM snapshot (<i>v4+</i>)
0x54	4	texMemSize	Byte count, 0x100000 (<i>v4+</i>)
0x58	4	mem1Size	Guest MEM1 size, retail 0x01800000 (<i>v5+</i>)
0x5C	4	mem2Size	Guest MEM2 size, retail 0x04000000 (<i>v5+</i>)
0x60	8	gameId	ASCII, NUL-padded; "00000000" if unknown (<i>v6+</i>)

Register block sizes count 32-bit words, **texMemSize** counts bytes. When reading an older file, zero-fill whatever it does not store and fall back to the retail memory sizes (before v5) and the default game ID (before v6).

3 Frame descriptor

One 64-byte entry per frame at **frameListOffset**.

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+10	+11	+12	+13	+14	+15
0x00	fifoDataOffset								fifoDataSize				fifoStart			
0x10	fifoEnd				memoryUpdatesOffset								numMemoryUpdates			
0x20	reserved															

Offset	Size	Field	Description
0x00	8	fifoDataOffset	Absolute offset of this frame's raw FIFO bytes
0x08	4	fifoDataSize	FIFO byte count
0x0C	4	fifoStart	Guest physical address of the CP FIFO ring base at record time
0x10	4	fifoEnd	Guest physical address of the CP FIFO ring end, has to be above fifoStart
0x14	8	memoryUpdatesOffset	Absolute offset of the memory-update table
0x1C	4	numMemoryUpdates	Entries in that table

fifoData is the GX command stream for one frame, with one twist: display lists are flattened. **CALL_DL** (0x40) never shows up, the contents of the list are recorded inline at the call site. A frame ends right at the end of the EFB copy command (BP 0x52) that presents it, with no trailing bytes. Dolphin's frame analyzer asserts this. Feeding the whole buffer produces exactly one presented frame. **fifoStart** and **fifoEnd** are not just informational. Dolphin programs the emulated CP ring from them and derives the watermarks from their distance, so they need plausible values.

4 Memory update

The FIFO references guest RAM by physical address (textures, vertex arrays, display lists, XF loads). Each frame comes with a table of RAM writes that have to land before the FIFO consumes them. One entry is 24 bytes:

	+0	+1	+2	+3	+4	+5	+6	+7
0x00	fifoPosition				address			
0x08	dataOffset							
0x10	dataSize				type	padding		

Offset	Size	Field	Description
0x00	4	fifoPosition	Byte offset into this frame's <code>fifoData</code> . Apply the write before feeding bytes at or past this offset
0x04	4	address	Guest physical destination address
0x08	8	dataOffset	Absolute offset of the payload in the file
0x10	4	dataSize	Payload byte count
0x14	1	type	See below

Value	Type
0x01	TEXTURE_MAP
0x02	XF_DATA
0x04	VERTEX_STREAM
0x08	TMEM

The type says what the data feeds, nothing more. Every kind is applied the same way: copy `data` to `address` in guest RAM. A replayer can use `TEXTURE_MAP` to invalidate texture caches that overlap the write. gecko decodes unknown values as `TEXTURE_MAP`.

Entries have to be sorted by `fifoPosition`. Dolphin walks the table in order and never sorts it. The gecko reader sorts on load anyway, just in case. Writers deduplicate by diffing every referenced range against a shadow copy of guest RAM and only emit an update when the bytes actually changed. A static texture shows up once in the whole file, not once per frame.

5 Replay

1. Validate `fileId` and `minLoaderVersion`.
2. Load `bpMem`, `cpMem`, `xfMem` and `xfRegs` into the GPU state and `texMem` into TMEM. This stands in for every register write the game did before the recording started. When replaying `bpMem` as register writes, skip the registers with side effects: PE finish (`0x45`), the PE tokens (`0x47`, `0x48`), the EFB copy trigger (`0x52`), the TLUT DMA trigger (`0x65`) and the write mask (`0xFE`). Dolphin and gecko both do this.
3. For each frame, interleave updates and FIFO data:

```
pos = 0
for u in sort_by(updates, fifo_position):
    p = min(u.fifo_position, len(fifo_data))
    feed(fifo_data[pos..p])          # GX command stream
    pos = p
    ram[u.address .. u.address+len(u.data)] = u.data
    feed(fifo_data[pos..])
present()                          # frame ends at its XFB copy
```

State carries across frames. A frame assumes the register and RAM state left behind by the frames before it, so frames cannot be skipped, only played from the start up to a target.

6 Versions

Version	Change
1	Base format: header, register blocks, frames, memory updates
2	EFB copy recording fixed. Dolphin treats v1 files as having broken EFB copies
4	<code>texMem</code> snapshot added
5	<code>mem1Size</code> / <code>mem2Size</code> added
6	<code>gameId</code> added

`minLoaderVersion` decides forward compatibility, not `fileVersion`. A writer that only appends optional data keeps `minLoaderVersion` low so old loaders still accept the file.